BACHUPALLY

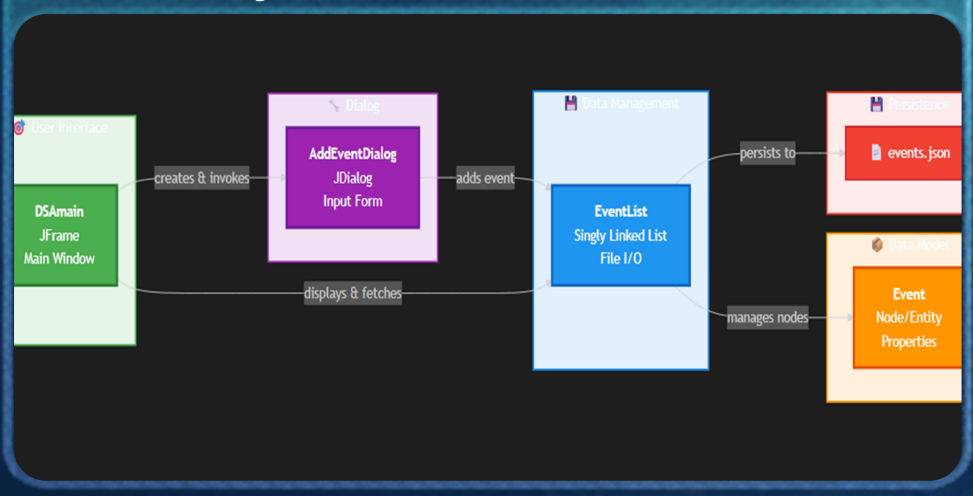
Personal Event Scheduler and Tracker

PROJECT BASED LEARNING FROM DSA

Abstract

Personal Event Scheduler and Tracker is a Java-based desktop application for event management using Data Structures and Algorithms. It implements Singly Linked List ADT for dynamic storage, Stack for undo operations, Queue for event processing, and Java Collections Framework. Users can add, view, sort, search, and categorize events with real-time display and demonstrates ADT design, linked list operations, stack/queue structures, and collections.

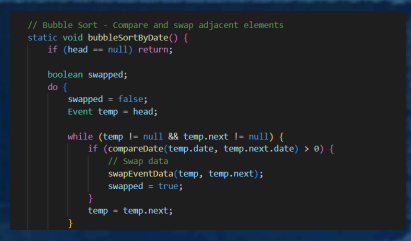
System Architecture



CO-WISE IMPLEMENTATION

C01 - Algorithm Analysis

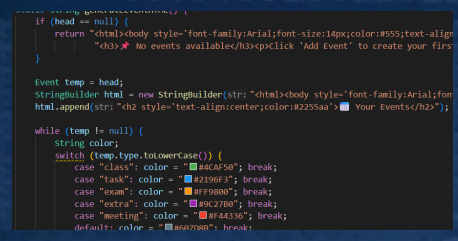
- Analyzed algorithms
- Compared sorts
- Measured complexity



```
// Bubble Sort - Compare and swap adjacent elements
static void bubbleSortByDate() {
    if (head == null) return;

    boolean swapped;
    do {
        swapped = false;
        Event temp = head;

        while (temp != null && temp.next != null) {
            if (compareDate(temp.date, temp.next.date) > 0) {
                // Swap data
                swapEventData(temp, temp.next);
                swapped = true;
            }
            temp = temp.next;
        }
```

CO2 - Arrays and Linked Lists

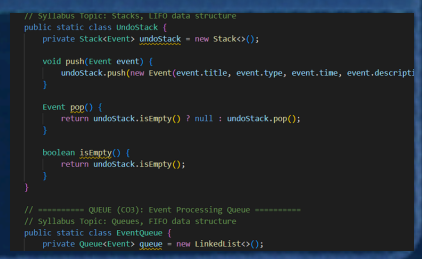
- Designed ADTs
- Used arrays
- Linked lists
- Modified structures



```
    if (head == null) {
        return "<html><body style='font-family:Arial;font-size:14px;color:#555;text-align
                "<h3>📌 No events available</h3><p>Click 'Add Event' to create your firs
    }

    Event temp = head;
    StringBuilder html = new StringBuilder(str: "<html><body style='font-family:Arial;fon
    html.append(str: "<h2 style='text-align:center;color:#2255aa'>📅 Your Events</h2>");

    while (temp != null) {
        String color;
        switch (temp.type.toLowerCase()) {
            case "class": color = "🟪#4CAF50"; break;
            case "task": color = "🟪#2196F3"; break;
            case "exam": color = "🟪#FF9800"; break;
            case "extra": color = "🟪#9C27B0"; break;
            case "meeting": color = "🟪#F44336"; break;
            default: color = "🟪#607D8B"; break;
```

C03 - Stack and Queue

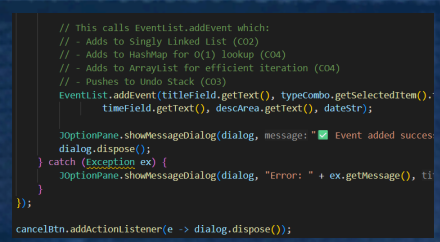
- Used stacks
- Used queues
- Deques implemented
- Built heaps



```
// Syllabus Topic: Stacks, LIFO data structure
public static class UndoStack {
    private Stack<Event> undoStack = new Stack<>();

    void push(Event event) {
        undoStack.push(new Event(event.title, event.type, event.time, event.descripti
    }

    Event pop() {
        return undoStack.isEmpty() ? null : undoStack.pop();
    }

    boolean isEmpty() {
        return undoStack.isEmpty();
    }
}

// ========= QUEUE (CO3): Event Processing Queue =========
// Syllabus Topic: Queues, FIFO data structure
public static class EventQueue {
    private Queue<Event> queue = new LinkedList<>();
```

C04 - Hash Data Structures

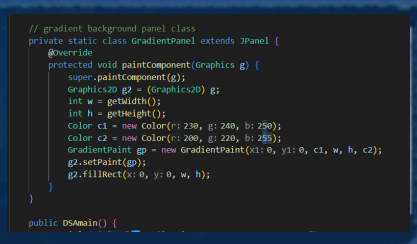
- Made hash tables
- Used Collections
- Lookup solutions
- Used Maps



```
        // This calls EventList.addEvent which:
        // - Adds to Singly Linked List (CO2)
        // - Adds to HashMap for O(1) lookup (CO4)
        // - Adds to ArrayList for efficient iteration (CO4)
        // - Pushes to Undo Stack (CO3)
        EventList.addEvent(titleField.getText(), typeCombo.getSelectedItem().
                timeField.getText(), descArea.getText(), dateStr);

        JOptionPane.showMessageDialog(dialog, message: "✅ Event added succes
        dialog.dispose();
    } catch (Exception ex) {
        JOptionPane.showMessageDialog(dialog, "Error: " + ex.getMessage(), ti
    }
});

cancelBtn.addActionListener(e -> dialog.dispose());
```

C05 - Applications of Linear Data Structures

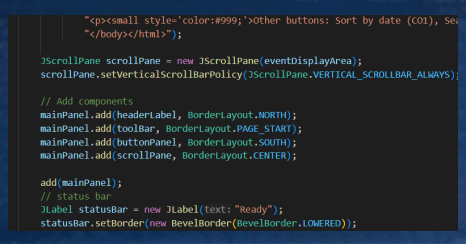
- Built with structures
- Practical uses
- DSA applications
- Managed data



```
// gradient background panel class
private static class GradientPanel extends JPanel {
    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        Graphics2D g2 = (Graphics2D) g;
        int w = getWidth();
        int h = getHeight();
        Color c1 = new Color(r: 230, g: 240, b: 250);
        Color c2 = new Color(r: 200, g: 220, b: 255);
        GradientPaint gp = new GradientPaint(x1: 0, y1: 0, c1, w, h, c2);
        g2.setPaint(gp);
        g2.fillRect(x: 0, y: 0, w, h);
    }
}

public DSAmain() {
```

C06 - Application Development Using DSA

- Java applications
- Coding skills
- Interactive software
- Project outcomes



```
            "<p><small style='color:#999;'>Other buttons: Sort by date (CO1), Sea
            "</body></html>");

JScrollPane scrollPane = new JScrollPane(eventDisplayArea);
scrollPane.setVerticalScrollBarPolicy(JScrollPane.VERTICAL_SCROLLBAR_ALWAYS);

// Add components
mainPanel.add(headerLabel, BorderLayout.NORTH);
mainPanel.add(toolBar, BorderLayout.PAGE_START);
mainPanel.add(buttonPanel, BorderLayout.SOUTH);
mainPanel.add(scrollPane, BorderLayout.CENTER);

add(mainPanel);
// status bar
JLabel statusBar = new JLabel(text: "Ready");
statusBar.setBorder(new BevelBorder(BevelBorder.LOWERED));
```

Contribution & Future Scope

Plannerz implements Singly Linked List ADT for event storage, Stack for undo operations, Queue for event processing, Bubble Sort for organization, and Linear Search for title-based retrieval. Future enhancements include advanced sorting algorithms, priority queues for notifications, database integration, web/mobile interfaces, calendar grid views

Plannerz

Name: D.Bala charan | Roll No. 2520090087 | Cluster: 2 | Section: 6